



亞洲大學
ASIA UNIVERSITY

Final Project Report Advanced Computer Programming

Topic

**Iris Flower Classification Using Machine
Learning**

Group : Group 5

Instructor : DINH-TRUNG VU

2025-06

Chapter 1 Introduction

1.1 Group Information

- 1) **GroupProjectRepository:**https://github.com/Linx25059/group-5/tree/main/Final_Project
- 2) **Group members:**
 1. 邱仁宏-109022066
 2. 陳逸凱-110021085
 3. 林哲熙-110022106

1.2 Overview

Scikit-Learn, train_test_split, K-Nearest Neighbors (KNN), Decision Tree, Random Forest, and classification metrics were used in this project. The objective of this project was to classify iris flowers using machine learning techniques. Experimental results showed that KNN, Decision Tree, and Random Forest all achieved 100% accuracy on the Iris dataset.

Chapter 2 Implementation

2.1 Class 1

KNeighborsClassifier.

2.1.1 Fields

n_neighbors = 5

2.1.2 Methods

fit()

predict()

2.1.3 Functions

K-Nearest Neighbors (KNN) is a supervised learning algorithm that classifies samples according to the majority vote of their nearest neighbors. The parameter n_neighbors determines the number of nearby samples considered during classification.

In this project, the KNN classifier was trained using the Iris dataset and then applied to predict the species of iris flowers in the testing set. The model successfully classified all testing samples with an accuracy of 100%.

2.2 Class 2

Class Name

DecisionTreeClassifier

Fields

`random_state = 42`

Methods

`fit()`

`predict()`

Functions

Decision Tree is a classification algorithm that makes decisions by splitting data according to feature values. The model forms a tree-like structure in which each node represents a decision condition and each branch represents a possible outcome.

The Decision Tree classifier was trained and evaluated using the Iris dataset. The model achieved an accuracy of 100%, indicating excellent classification performance.

2.3 Method/Function 1

Function Name

`train_test_split()`

Description

The `train_test_split()` function was used to divide the dataset into training and testing subsets. In this project, 80% of the data were used for training and 20% for testing.

Separating training and testing data is important because it allows the model to be evaluated on unseen samples. This process helps determine whether the model can generalize well to new data rather than simply memorizing the training set.

2.4 Method/Function 2

Function Name

`classification_report()`

Description

The `classification_report()` function generates a detailed evaluation report for classification models. It provides Precision, Recall, F1-score, and Support values for each class.

These evaluation metrics offer a more comprehensive understanding of model performance than accuracy alone. In this project, the classification report indicated perfect performance for all iris flower categories.

Chapter 3 Results

3.1 Result 1

Accuracy Comparison

Model	Accuracy
-------	----------

KNN	100%
-----	------

Decision Tree	100%
---------------	------

Random Forest	100%
---------------	------

Discussion

All three machine learning models achieved perfect accuracy on the Iris dataset. This result suggests that the dataset is well-structured and the classes can be effectively separated using different classification approaches.

3.2 Result 2

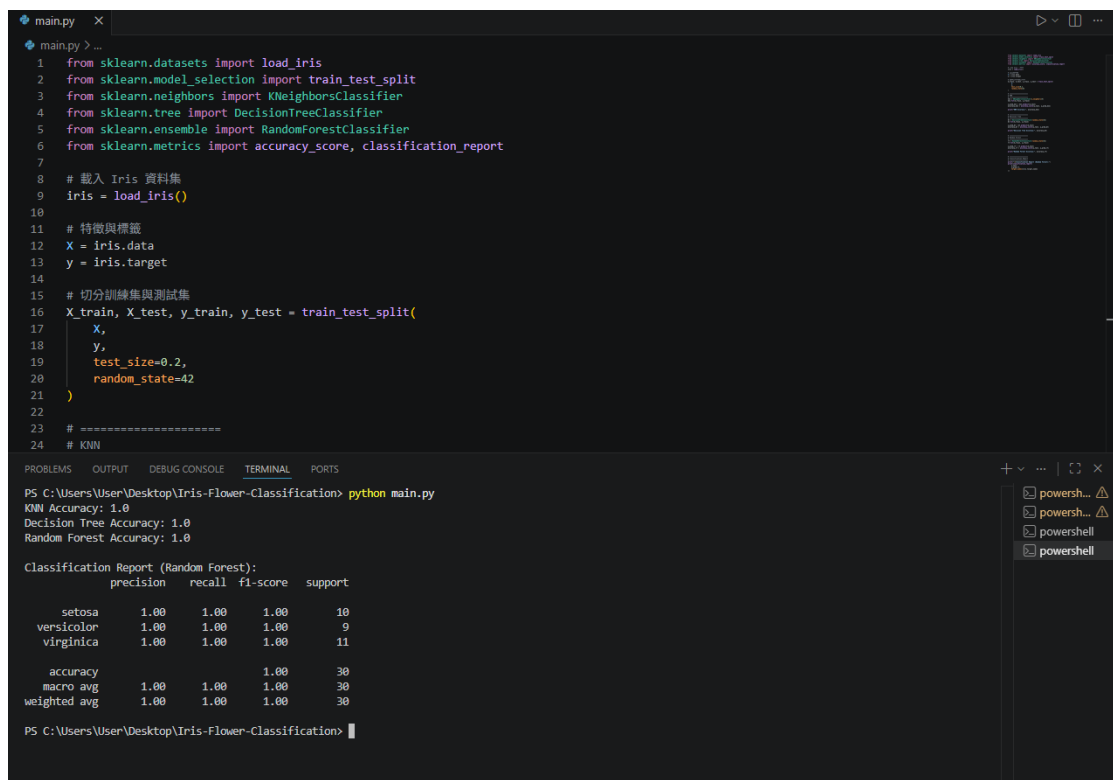
Classification Report :

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11
accuracy			1.00	30

macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Discussion

The classification report showed that all evaluation metrics reached 1.00. This means that the models correctly identified every testing sample in this experiment.



```
main.py X
main.py > ...
1 from sklearn.datasets import load_iris
2 from sklearn.model_selection import train_test_split
3 from sklearn.neighbors import KNeighborsClassifier
4 from sklearn.tree import DecisionTreeClassifier
5 from sklearn.ensemble import RandomForestClassifier
6 from sklearn.metrics import accuracy_score, classification_report
7
8 # 載入 Iris 資料集
9 iris = load_iris()
10
11 # 特徵與標籤
12 X = iris.data
13 y = iris.target
14
15 # 切分訓練集與測試集
16 X_train, X_test, y_train, y_test = train_test_split(
17     X,
18     y,
19     test_size=0.2,
20     random_state=42
21 )
22
23 # =====
24 # KNN
25
26 PS C:\Users\User\Desktop\Iris-Flower-Classification> python main.py
KNN Accuracy: 1.0
Decision Tree Accuracy: 1.0
Random Forest Accuracy: 1.0
Classification Report (Random Forest):
      precision    recall  f1-score   support
setosa         1.00      1.00      1.00        10
versicolor    1.00      1.00      1.00         9
virginica      1.00      1.00      1.00        11
accuracy              1.00        30
macro avg       1.00      1.00      1.00        30
weighted avg    1.00      1.00      1.00        30
PS C:\Users\User\Desktop\Iris-Flower-Classification>
```

Chapter 4 Conclusions

This project successfully implemented machine learning classification using the Iris dataset and the Scikit-Learn library. Three classification models, including KNN, Decision Tree, and Random Forest, were evaluated and compared.

The results showed that all models achieved 100% accuracy on the testing dataset. Through this project, we learned how to prepare data, train machine learning models, evaluate their performance, and interpret classification results.

In future work, larger and more complex datasets could be used to further investigate the strengths and limitations of these machine learning algorithms.